

Software Requirements Specification for
AlumTrak: JHCSC BSIT Alumni Employment Tracking and Reminder System



In Partial Fulfillment
of the Requirements for the Subject
IT 305 – Integrative Programming and Technologies 2
Bachelor of Science in Information Technology

By:

Bisnar, Quien B.
Marchan, Marian G.
Oliveros, Josie G.

December 2025

Table of Contents

1 Introduction	6
1.1 Purpose.....	6
1.2 Intended Audience	6
1.3 Product Scope	6
1.3.1 Included Functionalities	7
1.3.2 Excluded Functionalities	8
1.4 Definitions, Acronyms, and Abbreviations.....	9
2 Overall Description	10
2.1 User Characteristics.....	10
2.2 Constraints.....	11
2.2.1 Technical Constraints	11
2.3 Assumptions and Dependencies.....	12
2.3.1 Assumptions	12
2.3.2 Dependencies	12
3 Specific Requirements.....	13
3.1 Functional Requirements	13
3.1.1 User Registration and Authentication	13
3.1.2 Alumni Profile Management.....	13
3.1.3 Document Management and Verification	13

3.1.4 Administrative Dashboard and Control	14
3.1.5 Alumni Dashboard and Profile Management	14
3.1.6 Automated Notification System.....	14
3.1.7 Reporting and Analytics	14
3.1.8 System Administration and Security	15
3.2 Non-functional Requirements.....	15
3.2.1 Performance Requirements	15
3.2.2 Reliability and Availability Requirements	15
3.2.3 Security Requirements.....	15
3.2.4 Usability Requirements	15
3.2.5 Compatibility Requirements	16
3.2.6 Maintainability Requirements	16
3.3 External Interface Requirements.....	16
3.3.1 User Interface Requirements	16
3.3.2 Alumni User Interface	16
3.3.3 Administrator User Interface	16
3.3.4 Hardware Interface Requirements	17
3.3.5 Software Interface Requirements	17
3.3.6 Communication Interface Requirements.....	17
3.4 System Models	18
3.4.1 Use Case Diagram	18

3.4.2 Entity-Relationship Diagram	19
3.5 Database Design	19
3.5.1 Physical Database Schema	20
3.5.2 Data Dictionary	20
3.5.3 Relationships and Integrity Constraints	21
3.6 Implementation	22
3.6.1 Development Environment and Configuration	22
3.6.2 Technology Stack	23
3.6.3 System Architecture.....	24
3.6.4 Core Module Implementation.....	25
3.6.5 Compliance and Maintainability	26
3.6.6 Tools Used.....	26
3.7 Testing	27
3.7.1 Load Test Results	27
3.7.2 Stress Test Results.....	29
3.7.3 User Acceptance Testing.....	34
3.8 Deployment.....	36
3.8.1 Deployment Environment.....	36
3.8.2 System Configuration	36
3.8.3 Maintenance and Monitoring.....	36
4 Appendices	38

Appendix A: User Acceptance Testing Survey Instrument.....	38
Appendix B: Data Dictionary	41
B.1 User Authentication and Account Management	41
B.2 Alumni Profile, Academic, and Professional Records.....	42
B.3 Document Management	44
B.4 In-System Alerts and Communication	45
B.5 System Logs and Audit Trails.....	46
Appendix C: Software Performance and Data Loading Documentation.....	47
C.1 Database Loading	47
C.2 Performance Testing Execution	48
Appendix D: Beta Test Logs and Photographic Evidence.....	50
Appendix E: Documentation of Project Development and Defense	52
E.1 Project Roadmap, Technical Consultations, and Development Phase.....	52
E.2 Final System Defense Documentation	53

1 Introduction

1.1 Purpose

The AlumTrak system is designed to provide Josefin H. Cerilles State College (JHCSC) with a comprehensive solution for tracking the employment status of Bachelor of Science in Information Technology (BSIT) graduates. This web-based platform enables alumni to update their personal and professional information, submit related documents, and receive automated reminders. For administrators, it offers tools for monitoring compliance, verifying submissions, and generating reports. The system supports institutional reporting requirements and enhances alumni engagement through systematic tracking and communication.

1.2 Intended Audience

The intended audience for the AlumTrak system encompasses user groups with distinct roles and responsibilities:

1. BSIT Alumni: Individuals who have completed the BSIT program and are required to provide periodic updates regarding their employment status, career progression, and professional development.
2. BSIT Department Administrators: Faculty members, department heads, program coordinators, and administrative staff responsible for alumni relations, data verification, compliance monitoring, and institutional reporting.

1.3 Product Scope

The AlumTrak system operates within a clearly defined scope that includes specific functionalities while explicitly excluding unrelated features:

1.3.1 Included Functionalities

The system shall provide the following core functionalities:

1.3.1.1 Alumni User Functions

1. **Registration and Authentication:** Secure self-registration for alumni with email-based verification and notification.
2. **Profile Management:** Alumni ability to create, view, and update personal, educational, and professional details.
3. **Employment Information Submission:** Alumni can submit and update their employment status and details. This includes:
 - Reporting current employment status (e.g., Employed, Student, Non-Employed).
 - Providing employer details and position information.
4. **Document Upload:** Functionality to upload supporting employment verification documents, including Certificates of Employment, Business Certificates, or Certificates of Registration.
5. **Submission Tracking:** Alumni can view the status of their submitted employment information and documents.

1.3.1.2 Administrative Functions

1. **Administrative Analytics Dashboard:** A centralized interface providing high-level metrics, trends, and visualizations of alumni data and submission statistics.
2. **Alumni Records Management:** Comprehensive administrative tools for searching, filtering, viewing, and performing batch operations on alumni records.
3. **Submission Review and Verification:** A dedicated administrative workflow for reviewing alumni employment submissions.

4. **Reporting:** Automated generation of PDF reports for compliance and employment statistics utilizing the TCPDF library.
5. **Audit Trail:** System logging of all significant user and administrative activities for security and compliance review.

C System Functions

1. **Automated Notification System:** An email reminder subsystem, integrated via NotificationAPI, for deadlines, approvals, status updates, and system notifications.

1.3.2 Excluded Functionalities

The following capabilities are explicitly outside the scope of the AlumTrak system:

1. Social networking or peer-to-peer communication features among alumni.
2. Direct integration with external Human Resources Information Systems (HRIS) or enterprise platforms beyond the defined document upload procedure.
3. Real-time chat, instant messaging, or forum functionalities.
4. Native mobile application development; the system is designated as a web-only platform.
5. Any form of financial transaction processing, payment gateways, or billing systems.
6. Automated job matching, recruitment services, or career marketplace functionalities.
7. Advanced data analytics or predictive modeling beyond basic trend visualization.
8. Integration with social media platforms for login or data sharing.

1.4 Definitions, Acronyms, and Abbreviations

To ensure consistent understanding of technical terminology throughout this document, the following definitions, acronyms, and abbreviations are provided:

- **Alumni/Graduate:** An individual who has successfully completed the Bachelor of Science in Information Technology (BSIT) degree program at Josefina H. Cerilles State College (JHCSC) and is no longer enrolled as a student.
- **AlumTrak:** The proposed web-based system for tracking BSIT alumni employment updates and compliance.
- **Business Certificate:** An official document verifying an individual's ownership or management of a legitimate business.
- **Certificate of Employment (COE):** A formal document from an employer confirming an individual's employment.
- **Certificate of Registration (COR):** A document issued by an institution verifying a student's active enrollment.
- **Reminder System:** The automated component of AlumTrak that sends notifications to alumni.
- **Tracking:** The process of monitoring BSIT alumni employment status and career progression.

2 Overall Description

2.1 User Characteristics

The AlumTrak system is designed to accommodate two distinct user groups with specific characteristics, skill levels, and interaction patterns:

- **Alumni Users (BSIT Graduates):**
 - Typically, aged 21-40, employed in IT-related fields, self-employed, or pursuing further studies.
 - Possess basic computer literacy and familiarity with web applications.
 - Access the system intermittently – primarily when changing jobs or responding to reminders.
 - Motivation includes compliance with institutional requests and maintaining connection with alma mater.
- **Administrative Users (JHCSC BSIT Department):**
 - Department chairs, program coordinators, faculty, and administrative staff
 - Have intermediate technical skills for data management and system operation
 - Access the system regularly during work hours for monitoring and reporting
 - Responsible for data verification, compliance tracking, and report generation

2.2 Constraints

The development, deployment, and operation of the AlumTrak system are subject to several constraints that define the boundaries within which the system must operate:

2.2.1 Technical Constraints

1. Hardware Limitations

- The system must be accessible via standard computing devices.
- Stable internet connectivity for optimal performance.

2. Development Constraints

- Project timeline limitations requiring completion within one academic semester.
- Development team size and expertise limiting implementation complexity.
- Budgetary restrictions affecting hosting options, third-party service subscriptions, and advanced feature implementation.

3. Resource Constraints

- Development timeline limited to one academic semester.
- Development team consisting of three BSIT students with specific technical skill sets.
- Budgetary limitations affecting hosting options and third-party service selections.
- Dependence on existing JHCSC IT infrastructure for deployment and maintenance.

2.3 Assumptions and Dependencies

The successful implementation and operation of the AlumTrak system are contingent upon several assumptions and dependencies.

2.3.1 Assumptions

- Alumni will maintain valid email addresses and respond to system notifications.
- Administrative users will consistently perform verification and monitoring functions.
- Users have access to internet-connected devices with modern web browsers.

2.3.2 Dependencies

- External Services: NotificationAPI for email delivery.
- Development Environment: XAMPP with specific configurations and extensions.
- Policy Framework: Clear institutional policies governing data collection, retention, and usage.
- User Adoption: Sufficient alumni participation to ensure data completeness and system utility.

3 Specific Requirements

3.1 Functional Requirements

The functional requirements define the specific capabilities and behaviors that the AlumTrak system must provide.

3.1.1 User Registration and Authentication

- The system shall allow users to register using a valid email address and secure password. The system shall authenticate users through login credentials before access is granted.
- The system shall implement role-based access control, distinguishing between alumni users and administrative users with appropriate permission levels.

3.1.2 Alumni Profile Management

- The system shall allow alumni to maintain comprehensive profiles including personal information, contact details, and addresses.
- The system shall support the management of employment and academic status categories including employed, self-employed, unemployed, and pursuing further studies.

3.1.3 Document Management and Verification

- The system shall support the upload of verification documents in PDF format only with a maximum file size of 2MB per document.
- The system shall provide administrative interfaces for reviewing, approving, rejecting, or requesting revisions to uploaded documents, with comment functionality.
- The system shall maintain document status tracking with audit trails of all status changes and administrative actions.

3.1.4 Administrative Dashboard and Control

- The system shall provide an administrative dashboard displaying key metrics.
- The system shall enable administrators to search, filter, and sort alumni records.
- The system shall provide the ability to open or close submissions for users.

3.1.5 Alumni Dashboard and Profile Management

- The system shall provide alumni users with a personalized dashboard displaying their current profile status (Approved, Pending, or Rejected) with clear visual indicators.
- The system shall enable alumni to view recent activities including submission history, document uploads, status changes, and administrative feedback with chronological organization.
- The system shall display detailed rejection reasons when an alumni profile is rejected, including specific document deficiencies or information corrections required for resubmission.
- The system shall provide alumni with notifications showing all system-generated reminders, status updates, and administrative communications received.

3.1.6 Automated Notification System

- The system shall provide customizable email templates for various notification types, including update reminders, rejection, and approval of documents.
- The system shall integrate with NotificationAPI for reliable email delivery with tracking capabilities.

3.1.7 Reporting and Analytics

- The system shall generate standardized PDF reports using the TCPDF library, containing alumni employment statistics, summary report and detailed alumni list.
- The system shall support custom report generation with configurable filters.

3.1.8 System Administration and Security

- The system shall implement comprehensive audit logging of all significant system events including administrative actions.

3.2 Non-functional Requirements

The non-functional requirements specify the quality attributes and constraints that govern the system's operation.

3.2.1 Performance Requirements

- Document upload operations shall complete within 15 seconds for files up to 2MB in size depending on the internet connectivity.
- Report generation shall complete within 15 seconds for datasets containing up to 100,000 records.

3.2.2 Reliability and Availability Requirements

- The system shall maintain operational availability during institutional business hours (8:00 AM to 5:00 PM, Monday to Friday).

3.2.3 Security Requirements

- User passwords shall be encrypted using hashing algorithm with appropriate.
- The system shall implement role-based access control.

3.2.4 Usability Requirements

- Error messages shall be clear, actionable, and contextually appropriate.
- The system shall maintain consistent navigation patterns and interface elements throughout.

3.2.5 Compatibility Requirements

- The system shall be compatible with modern web browsers, including Google Chrome and Microsoft Edge.

3.2.6 Maintainability Requirements

- The system shall be developed using modular architecture with clear separation of concerns.

3.3 External Interface Requirements

3.3.1 User Interface Requirements

The AlumTrak system shall provide distinct, role-based user interfaces tailored to the specific needs and workflows of Alumni and Administrator users.

3.3.2 Alumni User Interface

- Dashboard: A personalized dashboard displaying current profile status (Approved, Pending, Rejected), and recent activities.
- Profile Management: Intuitive forms for updating personal information.
- Employment Information: Intuitive forms for updating employment and/or educational status.
- Notifications Panel: A centralized view of all system-generated reminders, status updates, and administrative communications with filtering options.
- Submission History: A chronological log of all previous profile updates, document submissions, and status changes with details of administrative feedback.

3.3.3 Administrator User Interface

- Administrative Dashboard: A comprehensive dashboard displaying key metrics including total alumni, pending and rejected submissions, graduation trends.

- Alumni Management Interface: Advanced search, filtering, and sorting capabilities for alumni records with batch operation support for document verification and status updates.
- Document Verification Workspace: A dedicated interface for reviewing, approving, rejecting, or requesting revisions to uploaded documents with comment functionality and audit trails.
- Generate Reports: Tools for generating, customizing, and exporting reports in PDF format with configurable filters and visualization options.
- Activity Log: A detailed, filterable log of all system events including administrative actions, user logins, profile changes, and document updates.

3.3.4 Hardware Interface Requirements

- Standard desktop or laptop computer with internet connection.

3.3.5 Software Interface Requirements

- Development Environment: XAMPP with Apache 2.4+, MySQL 8.0+, and PHP 8.0+, VS Code.
- Report Generation: TCPDF library (version 6.4+) for PDF creation.
- Email Service: NotificationAPI integration.
- Frontend Framework: Tailwind CSS utility classes with JavaScript.

3.3.6 Communication Interface Requirements

- All web communications shall use HTTPS.
- Email delivery shall support both API-based transmission methods.

3.4 System Models

3.4.1 Use Case Diagram

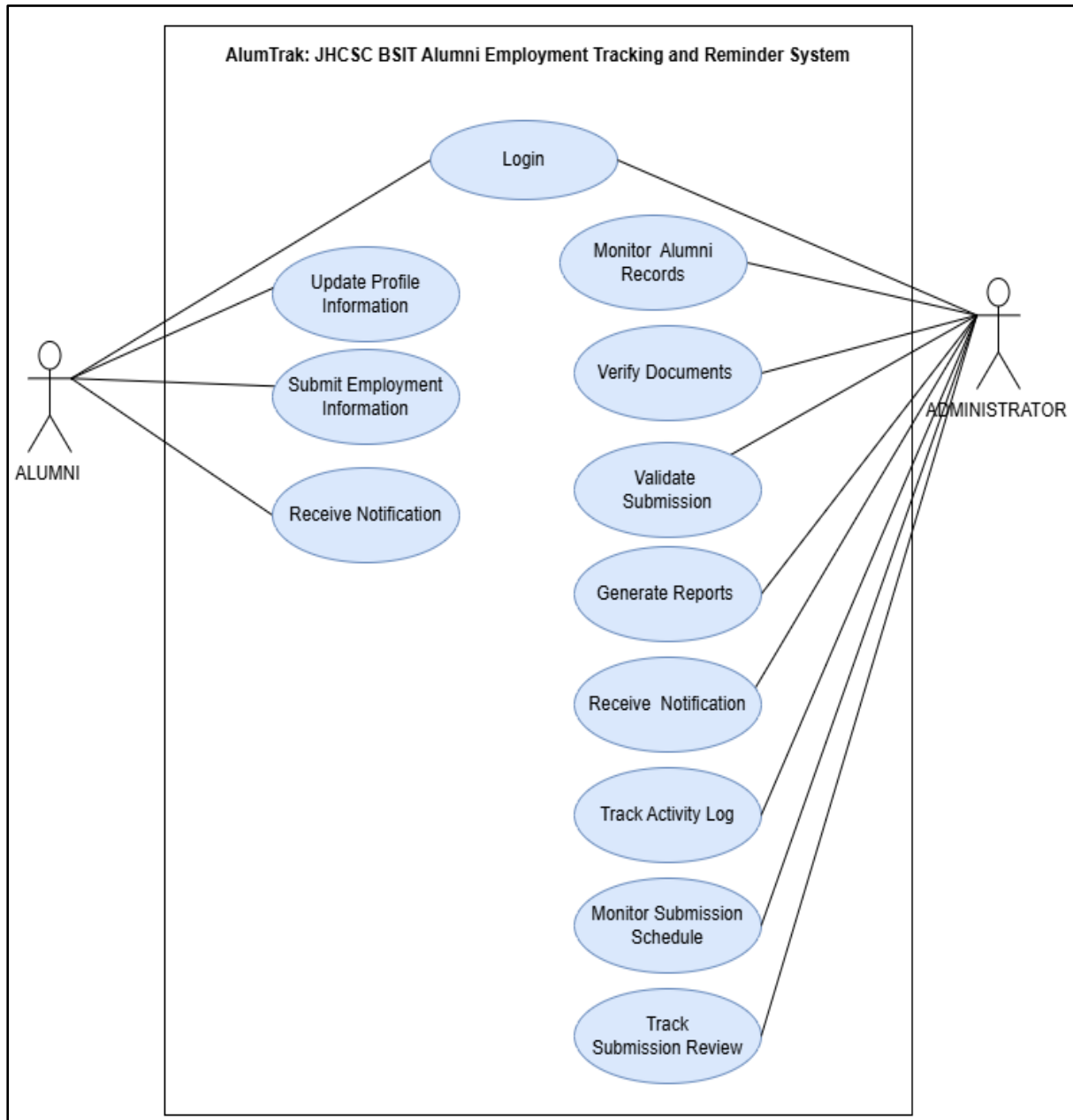


Figure 3.1. Use Case Diagram

3.4.2 Entity-Relationship Diagram

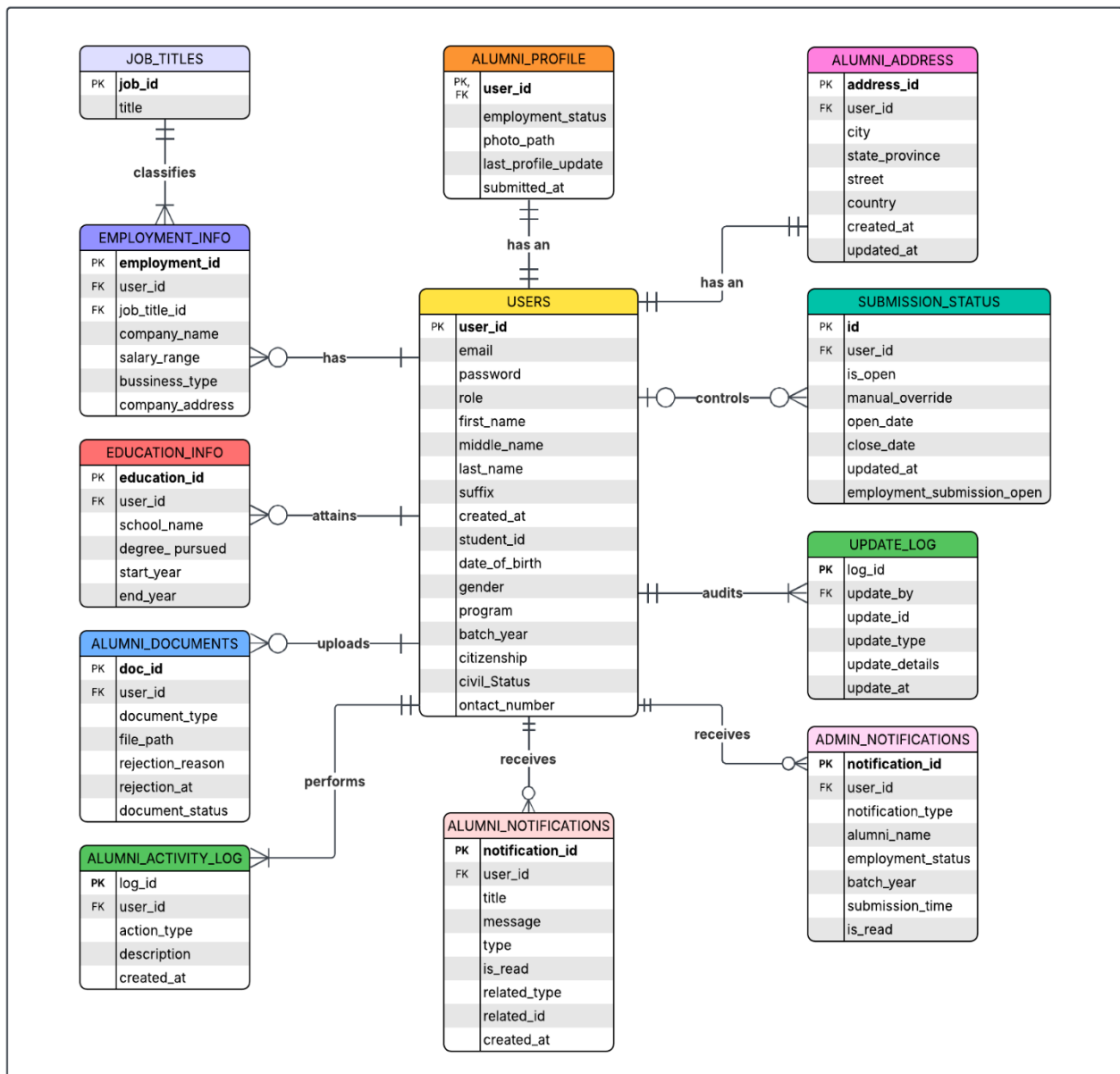


Figure 3.2. Entity-Relationship Diagram

3.5 Database Design

The AlumTrak system utilizes a relational database management system, MariaDB 10.4.32, to ensure structured data storage and referential integrity. This section details the physical schema, specific data types, and the constraints that govern the interactions between entities.

3.5.1 Physical Database Schema

The figure below represents the actual implemented schema of the alumni_tracking database. It illustrates the physical relationships, primary keys, and foreign key connections established within the MariaDB engine.

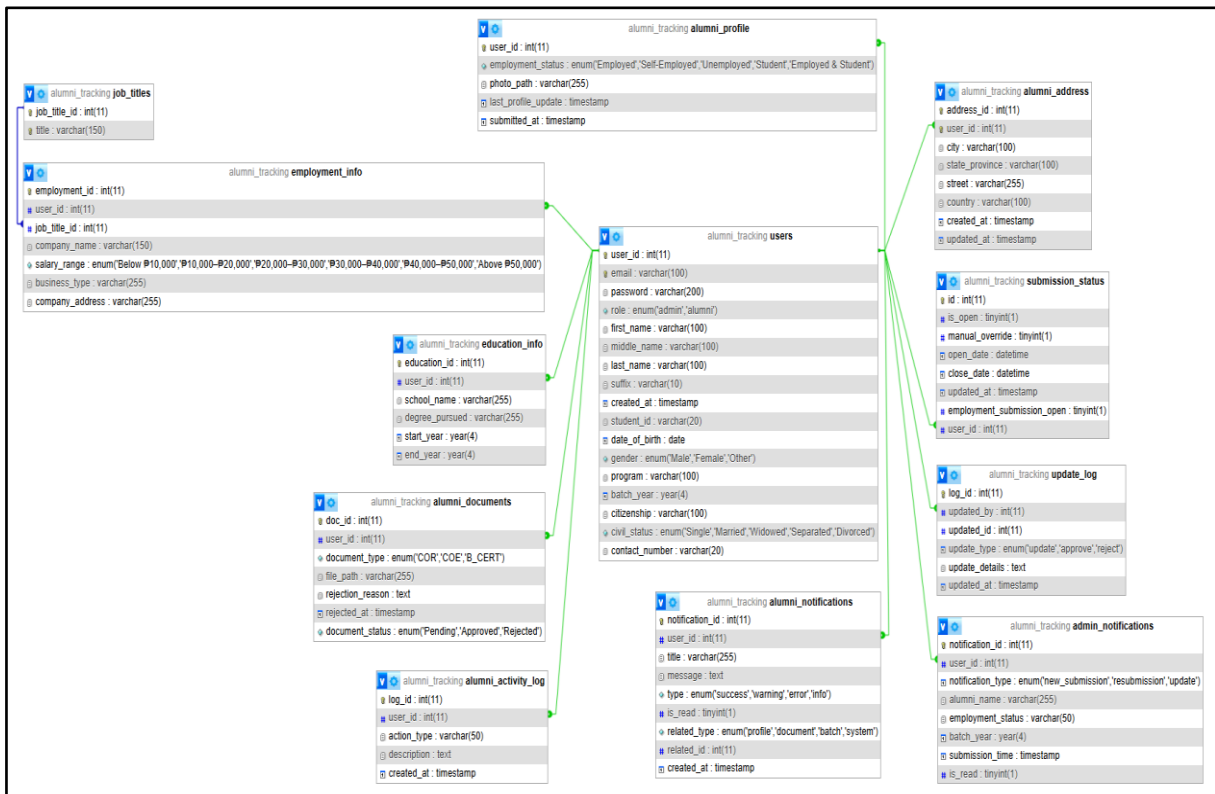


Figure 3.3. AlumTrak Physical Database Schema (MariaDB Designer View)

3.5.2 Data Dictionary

The system database consists of 12 core tables categorized into five functional groups. These structures ensure data integrity and support role-based access for Admins and Alumni. For the complete technical specifications, including data types, constraints, and primary and foreign key relationships, refer to **Appendix B**.

3.5.2.1 User Authentication and Account Management

Manages secure login credentials, Bcrypt-hashed passwords, and role-based access control.

3.5.2.2 Alumni Profile and Professional Records

Tracks academic history and career progress. This group utilizes the job_titles lookup table to ensure standardized reporting within the employment_info table.

3.5.2.3 Document Management

Handles the storage and verification status of uploaded institutional credentials (PDF format).

3.5.2.4 In-System Alerts

Manages automated notifications and communication triggers for both administrative updates and alumni activities.

3.5.2.5 System Logs and Audit Trails

Maintains a historical record of all user actions and administrative changes to ensure system accountability.

3.5.3 Relationships and Integrity Constraints

To ensure the stability and accuracy of the AlumTrak system, several database-level constraints were implemented:

1. **Referential Integrity via Foreign Keys:** The schema utilizes Foreign Key (FK) constraints with ON DELETE CASCADE actions. This ensures that the deletion of a primary record in the users table automatically purges associated data in dependent tables, preventing orphaned records.
2. **Lookup Table Normalization:** Employment data is standardized by linking employment_info to a dedicated job_titles table, preventing inconsistent entries and ensuring reliable institutional reporting.
3. **Data Consistency via ENUMs:** To maintain uniform datasets for accreditation, specific fields such as employment_status and salary_range are restricted to predefined ENUM values, eliminating the risk of invalid entries or varied spellings.

4. **Audit Trails:** Every critical interaction is tracked via `TIMESTAMP` fields. Tables such as `alumni_activity_log` and `update_log` use automatic timestamps to provide a non-repudiable history of system usage and administrative decisions.
5. **Session and Submission Synchronization:** The `submission_status` table functions as a global gatekeeper. By utilizing `manual_override` and `is_open` flags, the system programmatically synchronizes alumni access with institutional submission windows.
6. **Database Indexing for Performance Optimization:** To maintain performance as the database scales, strategic indexing was applied beyond default Primary Keys (PK):
 - **Unique Constraints:** Applied to `email` and `student_id` to prevent duplicate registrations.
 - **Composite Indexing:** To optimize complex queries, the system utilizes composite indexes such as `idx_user_created` (covering `user_id` and `created_at`) and `idx_user_read` (covering `user_id` and `is_read`). These allow the system to sort and filter notifications and logs simultaneously without additional processing overhead.
 - **Search and Filter Optimization:** Secondary Indexes are placed on frequently filtered columns, including `employment_status` in the `alumni_profile` table and `action_type` in the `alumni_activity_log`.
 - **Foreign Key Performance:** Indexes such as `fk_employment_alumni` and `fk_employment_job` are utilized to ensure efficient JOIN operations between core entities.

3.6 Implementation

3.6.1 Development Environment and Configuration

The system was developed within a standardized local environment using XAMPP as the integrated development stack and VS Code as the primary Integrated Development Environment. The technical configuration consisted of the following components:

- **XAMPP Development Stack**

- **Apache 2.4.58:** Functioned as the local web server to handle HTTP requests and host the application during development.
 - **MariaDB 10.4.32:** Utilized as the relational database management system (MySQL compatible) for structured data storage and alumni record management.
 - **PHP 8.2.12:** Served as the core server-side scripting language for developing system logic and database interactions.
 - **phpMyAdmin 5.2.1:** Used as the web-based interface for database administration and SQL query execution.
- **VS Code:** Employed as the primary IDE for writing, formatting, and debugging source code across the full development lifecycle.
 - **Version Control:** Managed using Git for local versioning and GitHub as the remote host for centralized collaboration and code backup.

3.6.2 Technology Stack

A technology stack was selected to meet functional and non-functional requirements:

- **Backend:** PHP 8.2.12 was chosen for its server-side scripting maturity, extensive library support, and seamless integration with the database.
- **Frontend:**
 - **HTML5:** Utilized for semantic document structuring to ensure accessibility and clear data hierarchy.
 - **CSS3:** styling was implemented via the Tailwind CSS framework, using a utility-first approach to create a responsive design.

- **Vanilla JavaScript:** Employed for core client-side interactivity and event handling.
- **Database:** MariaDB 10.4.32 provided a robust, relational data management system with support for transactions and referential integrity.
- **Libraries and APIs:**
 - **TCPDF:** Enabled server-side generation of structured PDF reports compliant with institutional formatting.
 - **Node.js/npm:** Used as the JavaScript runtime and package manager for managing the Tailwind CSS build process.
 - **NotificationAPI:** Handled reliable, templated email delivery for automated reminders and alerts.

3.6.3 System Architecture

AlumTrak follows a three-tier client-server architecture, separating concerns across presentation, application, and data layers:

1. **Presentation Tier:** Browser-rendered HTML5/CSS3/JS interfaces, dynamically generated by PHP scripts. Role-specific views (Alumni, Administrator) are served based on authenticated sessions.
2. **Application Tier:** PHP-based business logic layer residing on the Apache server. This tier processes requests, enforces business rules, manages authentication, and interfaces with the database and external APIs.
3. **Data Tier:** MariaDB database managing persistent storage. All data interactions occur via parameterized queries to ensure security and integrity.

3.6.4 Core Module Implementation

3.6.4.1 Authentication and Authorization System

A session-based login system was implemented using PHP's native session handling. Passwords are hashed using bcrypt. Role-Based Access Control (RBAC) distinguishes between:

- **Alumni** – can validate profiles, update employment information and upload documents, and view status.
- **Administrator (JHCSC BSIT Faculty)** – can verify documents, generate reports, manage alumni data.

3.6.4.2 Profile and Document Management Engine

Document uploads are restricted to PDF format and profile photos to a maximum file size of 2MB. To ensure file system integrity and security, all uploads are stored using system-generated filenames to prevent naming conflicts and directory traversal attacks. Administrators manage these submissions via a dedicated verification interface, which allows for review, approval, rejection, or request for revisions of alumni records.

3.6.4.3 Automated Notification Subsystem

Integrated with NotificationAPI, the system sends automated email reminders for updates, document approvals, rejections, and resubmissions, and compliance alerts.

3.6.4.4 Reporting and Visualization Modules

TCPDF generates structured PDF reports for compliance, employment statistics, and CHED/accreditation requirements.

3.6.4.5 Administrative Dashboard

A comprehensive dashboard provides faculty with tools to:

- Monitor compliance rates

- View graduation and employment status trends
- View audit logs

3.6.5 Compliance and Maintainability

The implementation aligns with the Philippines' Data Privacy Act (RA 10173) through data minimization, encryption of sensitive information, and comprehensive access logging. The codebase adheres to PSR-12 coding standards and includes inline PHPDoc comments to facilitate future maintenance and scalability. A modular structure allows for the replacement or upgrade of individual components (e.g., switching mapping APIs) with minimal systemic impact.

3.6.6 Tools Used

- **XAMPP** – Local server environment
- **PHPMyAdmin** – Database management
- **Visual Studio Code** – Primary code editor
- **Git & GitHub** – Version control and collaboration
- **Composer** – PHP dependency and package manager
- **TCPDF** - PDF report generation library
- **JMeter** - Software performance testing
- **Windows Terminal/Powershell** - Automation and scripting
- **NotificationAPI** - email notification and delivery service

This implementation ensures a secure, scalable, and user-friendly system tailored to the needs of JHCSC BSIT faculty and alumni.

3.7 Testing

The Alumtrak System is a web-based platform that manages alumni profiles, employment records, and scheduled notification reminders. In this system:

- A user refers to an authenticated client (alumni or administrator) performing actions through the system's web interface.
- A transaction refers to a complete request–response cycle, such as loading a page, retrieving alumni data, updating a profile, or executing an administrative status action.
- A load refers to the combined number of database records processed and the volume of concurrent users generating requests.

3.7.1 Load Test Results

The Alumni Employment Tracking and Reminder System is a web-based platform used by alumni and administrators to update profiles, track employment data, and manage automated reminders. The load test simulated increasing data volume to reflect real-world system growth and evaluated how database size affects system responsiveness during typical user interactions.

3.7.1.1 Metrics Measured

The evaluation tracked the following key performance metrics:

- **Database load:** Amount of stored records used to measure how increasing data volume impacts query execution and page response.
- **Average Response Time (ms):** Time required for the system to complete a request.
- **Throughput (requests/sec):** Number of requests the system can process per second.
- **System Performance Observation:** Qualitative interpretation of system behavior.

3.7.1.2 Test Environment

All load and stress tests were executed on a local machine environment, using JMeter as the load-testing tool. The database (MariaDB) and web server (Apache) were running on the same machine, which may increase latency under peak loads due to shared resource contention.

Table 3.1 *Load Test Performance Metrics*

Database load (records)	Average Response Time (milliseconds)	Throughput (requests/sec)	System Performance Observation
10,000	33	9.50391	Fast response time, high data received rate, and moderate throughput. Represents excellent performance for low-volume data. GUI is responsive.
50,000	116	8.5	Initial latency degradation is observed, suggesting the start of a database bottleneck. Throughput remains static. GUI remains responsive.
100,000	294	35.6	The system encounters severe latency & throughput jump. It's trying to cope by handling many requests at once, but each request is badly delayed.
200,000	288	57.6	The system achieves peak throughput and high latency persists.

The system demonstrates strong performance at low data volumes and progressively degrades as database size increases. At 10,000 records, response times are fast and the interface remains highly responsive. At 50,000 records, the first signs of latency appear, indicating emerging database I/O constraints, although throughput remains stable and the GUI still performs well. At 100,000 records, response times rise sharply and throughput spikes, showing that the system is attempting to process more requests concurrently but is

increasingly delayed per request. At 200,000 records, throughput reaches its highest level, but latency remains significantly elevated, confirming that the system is under sustained heavy load and operating near or at its performance limits.

3.7.2 Stress Test Results

The stress test simulated heavy concurrent user activity to replicate peak usage scenarios, such as mass profile updates or administrative validation periods. The goal was to determine how the system behaves as user load increases beyond normal operating levels, identify the point at which performance degradation or failures occur, and assess how effectively the system recovers once the load is reduced back to normal operating conditions.

3.7.2.1 Metrics Measured

The stress test focused on evaluating performance under growing user pressure using the following metrics:

- **Database load:** Amount of stored records used to measure how increasing data volume impacts query execution and page response.
- **Concurrent Users:** The number of simultaneous users performing actions, used to measure system stability as load increases beyond normal levels.
- **User Actions:** Specific operations executed by each virtual to simulate real interaction patterns.
- **Average System Response Time (ms):** Measures how long each user action takes to complete.
- **Overall Average Response Time (ms):** A combined metric representing the mean latency across all executed actions in a given test scenario.
- **Error Rate (%):** Indicates the percentage of failed, timed-out, or incomplete requests. A rising error rate signals that the system is exceeding its processing capacity.

- **Observation and Failure Mode Analysis:** Qualitative notes identifying visible bottlenecks and system slowdowns.

Table 3.2 *Stress Test Performance, Breaking Point Analysis, and System Recovery*

Databa se Load	Concurre nt Users	User Actions	Average System Respon se (ms)	Overall Average System Respon se (ms)	Over all Error Rate Rate (%)	Observati on & Failure Mode
150,000	15	View_Login_Page	12	19	0.00	Excellent performance; the system is stable and well-performing under this low-to-moderate load.
		Login_Action	21			
		Alumni_Dashboard	14			
		View_Profile_Page	16			
		Update_Profile_Action	35			
		Admin_Update_Status_Action	16			
150,000	25	View_Login_Page	27	50	0.00	Acceptable performance with bottleneck; the system is functionally stable but shows unacceptable latency for the slowest requests.
		Login_Action	65			
		Alumni_Dashboard	37			
		View_Profile_Page	41			
		Update_Profile_Action	94			
		Admin_Update_Status_Action	35			
150,000	50	View_Login_Page	30	60	0.00	High latency but

		Login_Action	65			functional; the system is nearing its capacity but has not yet reached its functional breaking point.
		Alumni_Dashboard	43			
		View_Profile_Page	48			
		Update_Profile_Action	119			
		Admin_Update_Status_Action	56			
150,000	100	View_Login_Page	719	66	0.77	Breaking point reached; the error rate confirms the system has exceeded its capacity, resulting in failed or timed-out transactions.
		Login_Action	1576			
		Alumni_Dashboard	920			
		View_Profile_Page	1114			
		Update_Profile_Action	1839			
		Admin_Update_Status_Action	1028			
Recovery test (return to 100,000 records)	Recovery test (return to 20 concurrent users)	View_Login_Page	14	29	0.00	The system successfully handles sustained load with no degradation under current testing conditions.
		Login_Action	39			
		Alumni_Dashboard	32			
		View_Profile_Page	29			
		Update_Profile_Action	39			
		Admin_Update_Status_Action	20			

The stress test shows that the system performs reliably at moderate concurrency but degrades sharply once user load approaches its upper limits. At 15 concurrent users, the system operates efficiently with very low response times and no errors. At 25 users, performance remains acceptable but early bottlenecks appear, especially during heavier actions such as profile updates. At 50 users, latency increases further, and although the system remains functional, it is clearly nearing maximum capacity. At 100 users, the system reaches its breaking point: response times surge into the high hundreds to thousands of milliseconds, and errors begin to appear (0.77% failure rate), indicating timeouts and unhandled load. The recovery test confirms that once load is reduced back to 20 users and 100,000 records, the system stabilizes and returns to normal response times, demonstrating successful recovery after stress conditions.

3.7.2.2 JMeter Stress Test Analysis: Response Time Graphs Across Varying User Loads

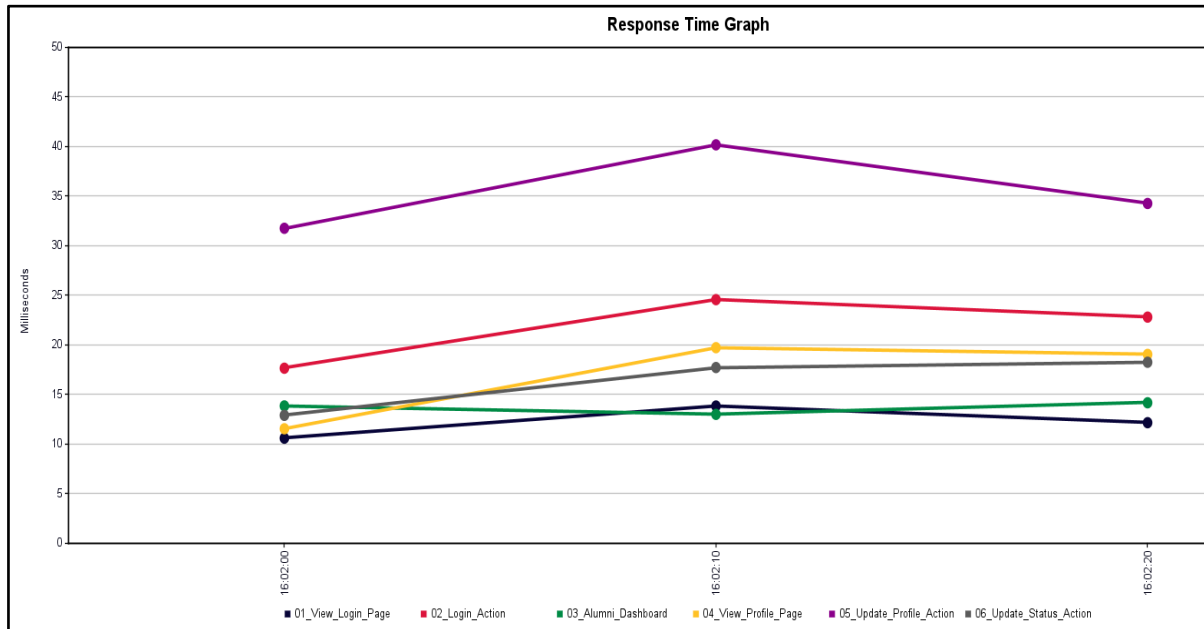


Figure 3.4. Peak system response times for 15 concurrent users: minimal latency and stable performance

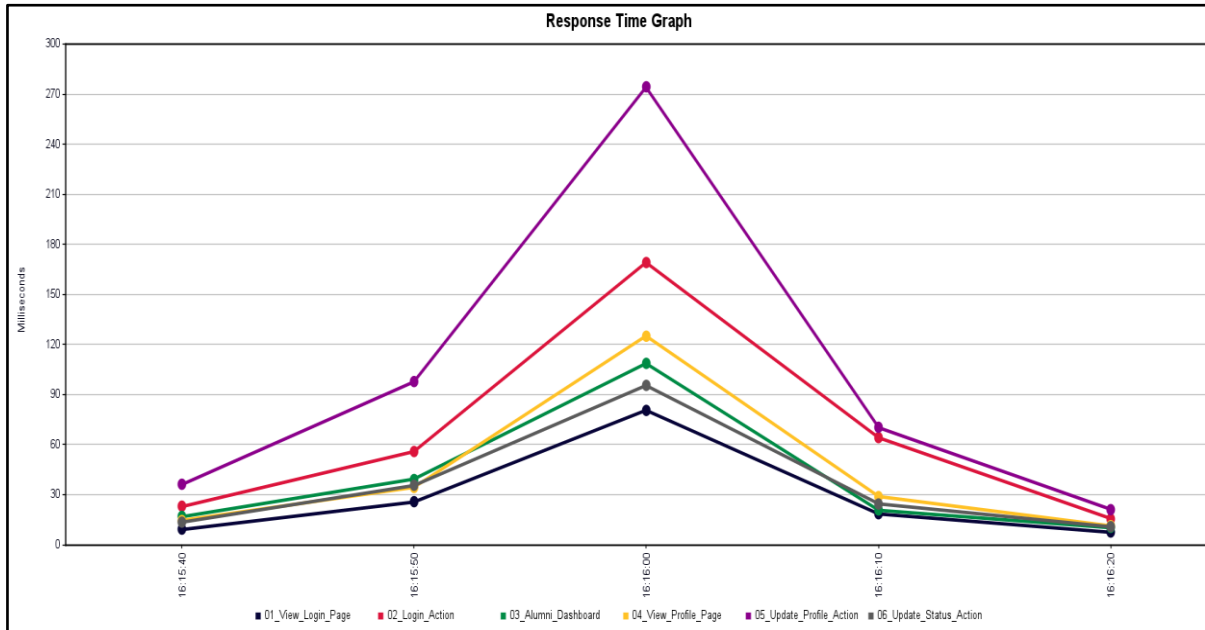


Figure 3.5. Peak system response times for 25 concurrent users: slightly higher latency, but stable performance

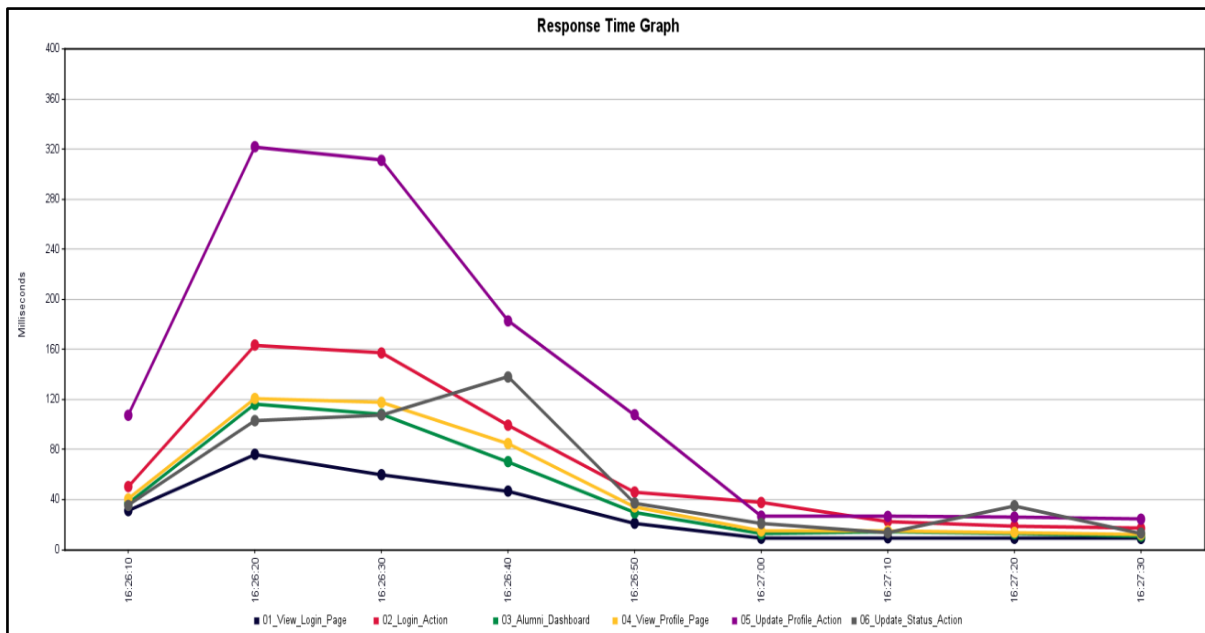


Figure 3.6. Peak system response times for 50 concurrent users: higher latency, nearing functional limits, but operational

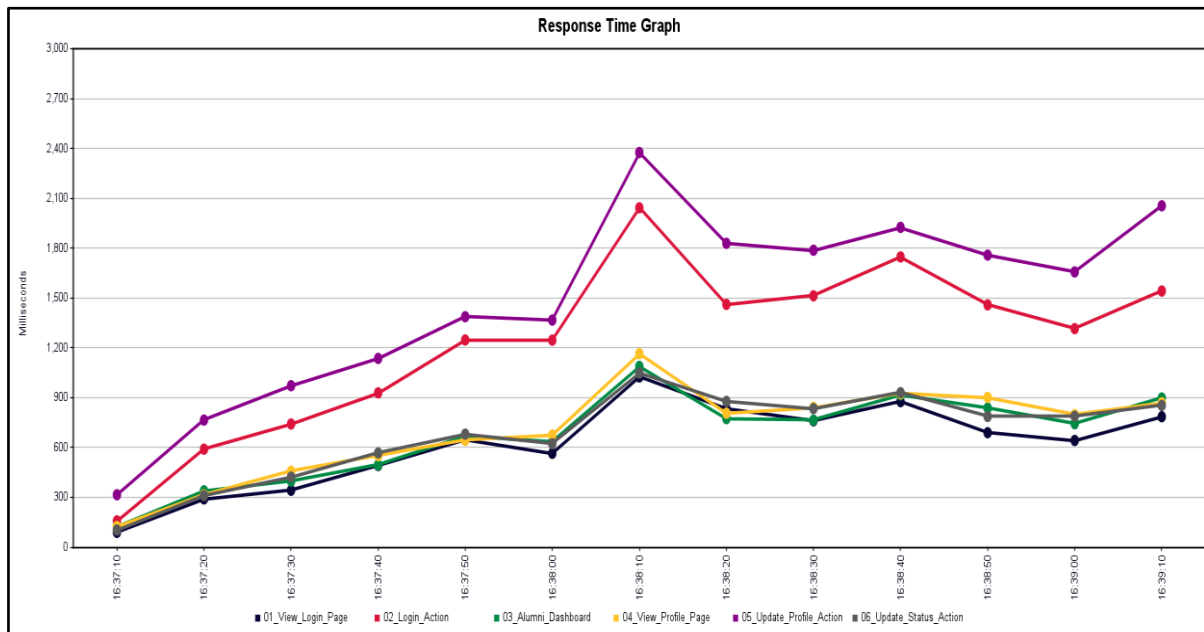


Figure 3.7. Peak system response times for 100 concurrent; breaking point reached with high latency and increased errors

3.7.3 User Acceptance Testing

User Acceptance Testing (UAT) is the final testing phase where the Alumni Employment Tracking and Reminder System is evaluated by intended users, including alumni and administrator, to ensure it meets functional and real-world operational requirements. Users test key features such as profile management, employment updates, document submission, and status validation to confirm correct functionality, usability, and data accuracy. Successful UAT indicates that the system is ready for deployment and meets both user and business requirements.

Table 3.3 *User Acceptance Testing Results*

Evaluation Dimension	Mean Score (1–4)	Percentage (%)	Verbal Interpretation
Usability	3.52	88%	Strong Agreement
Reliability	3.40	85%	Strong Agreement
Interface Design	3.48	87%	Strong Agreement
Overall System Evaluation	3.47	87%	Strong Agreement

The results indicate that the system achieved a high level of user acceptance across all evaluation dimensions. Usability obtained the highest rating at 88%, suggesting that respondents found the system easy to learn and operate. Interface Design followed closely at 87%, reflecting a generally positive perception of the system’s layout and navigational clarity. Reliability recorded an 85% agreement rate, indicating that users largely trust the system’s stability and performance. Overall, the system achieved an 87% approval rating, which corresponds to a Strong Agreement interpretation.

For a detailed breakdown of individual responses, the raw data is available via:
<https://tinyurl.com/UATDataqn>



Figure 3.8. *QR Code for AlumTrak User Acceptance Testing (UAT) Raw Data*

3.8 Deployment

The AlumTrak system is deployed using a local server environment configured for development and institutional use. Deployment begins by installing and configuring the required software stack, ensuring that the system operates securely and reliably.

3.8.1 Deployment Environment

The system is deployed on a server machine using the following components:

- Apache Web Server for handling HTTP requests
- PHP for server-side processing
- MariaDB for database management
- XAMPP as the integrated server environment

The application files are placed in the server's web directory, and the system is accessed through a web browser using the server's IP address and assigned port number.

3.8.2 System Configuration

- Database schemas and tables are created using MariaDB based on the system design.
- Configuration files are set to establish database connections and environment variables.
- Role-based access control is enabled to separate alumni and administrator privileges.
- Email notification services are configured using Notification API.

3.8.3 Maintenance and Monitoring

- Regular database backups are performed to prevent data loss

- System logs are monitored to detect errors, performance issues, and unauthorized access
- Periodic updates are applied to PHP libraries and dependencies
- Performance testing is conducted after major updates to ensure system stability
- Administrators manage user accounts, verify data integrity, and oversee system usage

4 Appendices

Appendix A: User Acceptance Testing Survey Instrument

Target Respondents: Bachelor of Science in Information Technology (BSIT) Students

Participant Profile: 1st Year, 2nd Year, 3rd Year (Block B), and 4th Year

Instrument Description: This instrument was was administered via Google Forms to collect quantitative and qualitative data regarding user experience. It consists of Likert-scale questions across three technical domains and a final section for open-ended feedback.

Response Range: 4 – Strongly Agree; 3 – Agree; 2 – Disagree; 1 – Strongly Disagree

Section 1: Usability				
<i>This section focuses on how easy and efficient it is for users to use the web-based system, including navigation, layout, and overall user experience.</i>	4	3	2	1
1. I found the system simple and easy to use.				
2. I observed that the features of the system worked well together.				
3. I believe that most users would learn to use the system quickly.				
4. I felt comfortable and confident while using the system.				
5. I found it easy to understand how each function of the system works.				
Section 2: Reliability	4	3	2	1

<i>This measures the consistency and dependability of the web-based system in performing its intended functions without errors or interruptions.</i>				
1. I observed that the system consistently functions as expected.				
2. I found the system stable and free from unexpected failures.				
3. I rarely experience inconsistent behavior when using the system.				
4. I noticed that the system operates smoothly without needing frequent technical assistance.				
5. I trust the system to work reliably every time I use it.				
Section 3: Interface <i>This refers to the design and layout of the web-based system, focusing on how easy and intuitive it is for users to interact with its features.</i>	4	3	2	1
1. I found the interface visually pleasant and easy to navigate.				
2. I received clear instructions from the interface whenever I made a mistake.				
3. I felt that the interface layout was clear and well-organized.				
4. I was able to find the information I needed quickly through the interface.				
5. I found it easy to recover from errors because the interface guided me well.				

Section 4: Suggestions for Improvement

This section allows participants to share ideas or recommendations to enhance the system's features, usability, and overall performance.

1. What is your overall impression of the system so far?
2. Did you encounter any difficulties while navigating the system? Please describe.
3. Were there any features that you expected but could not find?
4. What improvements or additional features would you like to see in the system?
5. Any other feedback or comments you'd like to share about the system?

Appendix B: Data Dictionary

This appendix provides a technical reference for the AlumTrak database, detailing each table's fields, data types, lengths, and constraints (PKs and FKs). Tables are grouped by function to ensure data integrity and clarify the system's backend structure.

B.1 User Authentication and Account Management

Table B.1 *Users Table (users)*

Column Name	Data Type	Constraint	Description
user_id	INT(11)	PK, AUTO INCREMENT, NOT NULL	Unique identifier for each account
email	VARCHAR(100)	UNIQUE, NOT NULL	Primary login credential
password	VARCHAR(200)	NOT NULL	Bcrypt-hashed password
role	ENUM('admin','alumni')	NOT NULL	Role-based access
first_name	VARCHAR(100)	NULL	First name
middle_name	VARCHAR(100)	NULL	Middle name (if any)
last_name	VARCHAR(100)	NULL	Last name
suffix	VARCHAR(10)	NULL	Name suffix (e.g., Jr., Sr., III)
created_at	TIMESTAMP	NOT NULL, default current_timestamp()	Time of creation
student_id	VARCHAR(20)	UNIQUE	Institutional ID number
date_of_birth	DATE	NULL	Birth date
gender	ENUM('Male','Female','Other')	NULL	Gender identity
program	VARCHAR(100)	NULL, default 'BSIT'	Academic program enrolled in
batch_year	YEAR(4)	NOT NULL	Year of graduation
citizenship	VARCHAR(100)	NOT NULL, default 'Filipino'	User's citizenship
civil_status	ENUM('Single','Married','Widowed')	NOT NULL, default 'Single'	Marital status (Default: Single)

	dowed','Separated','Divorced')		
contact_number	VARCHAR(20)	NULL	Usre's phone or mobile number.

Table B.2 *Submission Status (submission_status)*

Column Name	Data Type	Constraint	Description
id	INT(11)	PK, AUTO INCREMENT	Unique status configuration ID.
user_id	INT(11)	FK	Link to users table.
is_open	TINYINT(1)	NULL. DEFAULT 0	Global toggle for profile updates.
manual_override	TINYINT(1)	NULL, DEFAULT 0	Flag indicating manual override of submission window (0 = no, 1 = yes).
open_date	DATETIME	NULLABLE	Scheduled start of submission window.
close_date	DATETIME	NULLABLE	Scheduled end of submission window.
updated_at	TIMESTAMP	NOT NULL	When the submission status was last updated
employment_subm ission_open	TINYINT(1)	NULL, DEFAULT 0	Indicates whether employment submissions are currently open (1 = open, 0 = closed)

B.2 Alumni Profile, Academic, and Professional Records

Table B.3 *Alumni Profile Summary (alumni_profile)*

Column Name	Data Type	Constraint	Description
user_id	INT(11)	PK, FK	References the corresponding record in the users table.
employment_status	ENUM ('Employed', 'Self-Employed', 'Unemployed', 'Student', 'Employed & Student')	NOT NULL	Current professional status
photo_path	VARCHAR(255)	NULL	Directory path for profile image.
last_profile_update	TIMESTAMP	NULL	Last profile update timestamp

submitted_at	TIMESTAMP	NOT NULL, default current_timesta mp()	When the profile was submitted
--------------	-----------	---	-----------------------------------

Table B.4 *Employment Information (employment_info)*

Column Name	Data Type	Constraint	Description
employment_id	INT(11)	PK, AUTO INCREMENT	Unique employment record ID.
user_id	INT(11)	FK	Link to users table.
job_title_id	INT(11)	FK	Lookup link to job_titles table.
salary_range	ENUM ('Below ₱10,000', '₱10,000– ₱20,000', '₱20,001– ₱30,000', '₱30,001– ₱40,000', '₱40,001 and above')	NULL	Alumni monthly salary range
business_type	VARCHAR(255)	NULL	Alumni business type
company_address	VARCHAR(255)	NULL	Address of company

Table B.5 *Educational History (education_info)*

Column Name	Data Type	Constraint	Description
education_id	INT(11)	PK, AUTO INCREMENT	Unique ID for educational records.
user_id	INT(11)	FK	Alumni user ID
school_name	VARCHAR(255)	NOT NULL	Name of institution
degree_pursued	VARCHAR(255)	NOT NULL	Post-graduate degree or certification.
start_year	YEAR(4)	NULL	Year started
end_year	YEAR(4)	NULL	Year finished

Table B.6 Alumni Address (*alumni_address*)

Column Name	Data Type	Constraint	Description
address_id	INT(11)	PK, AUTO INCREMENT	Unique address record ID.
user_id	INT(11)	FK	Link to users table.
city	VARCHAR(100)	NOT NULL	City of residence.
country	VARCHAR(100)	NOT NULL	Country of residence.
street	VARCHAR(255)	NULL	Street portion of the address.
state_province	VARCHAR(100)	NULL	State or province.
created_at	TIMESTAMP	NOT NULL, default current_timestamp()	When the address was created
updated_at	TIMESTAMP	NOT NULL, default current_timestamp() on update current_timestamp()	Last time the address was updated

Table B.7 Job Titles Lookup (*job_titles*)

Column Name	Data Type	Constraint	Description
job_title_id	INT(11)	PK, AUTO INCREMENT	Unique ID for job category.
title	VARCHAR(150)	NOT NULL	Official occupation title.

B.3 Document Management

Table B.8 Alumni Documents (*alumni_documents*)

Column Name	Data Type	Constraint	Description
doc_id	INT(11)	PK, AUTO INCREMENT	Unique document identifier
user_id	INT(11)	NULL	Alumni who owns the document
rejected_at	TIMESTAMP	NULL	Time when document was rejected
rejection_reason	TEXT	NULL	Reason for document rejection

document_type	ENUM('COR','COE','B_CERT')	NOT NULL	Type of document
file_path	VARCHAR(255)	NOT NULL	File path of submitted documents
document_status	ENUM('Pending','Approved','Rejected')	NULL, default 'Pending'	Verification status

B.4 In-System Alerts and Communication

Table B.9 Admin Notifications (*admin_notifications*)

Column Name	Data Type	Constraint	Description
notification_id	INT(11)	PK, AUTO INCREMENT, NOT NULL	Unique ID for each admin notification
user_id	INT(11)	NULL	ID of the user associated with the notification
notification_type	ENUM('new_submission','resubmission','update')	NOT NULL	Submission or Update alert.
alumni_name	VARCHAR(255)	NOT NULL	Name of the alumni related to the notification
is_read	TINYINT(1)	NULL	Acknowledgment status
employment_status	VARCHAR(50)	NOT NULL	Current employment status of alumni
batch_year	YEAR(4)	NULL	Alumni batch year
submission_time	TIMESTAMP	NOT NULL	Time when the notification was created

Table B.10 Alumni Notifications (*alumni_notifications*)

Column Name	Data Type	Constraint	Description
notification_id	INT(11)	PK, AUTO INCREMENT	Unique alumni alert ID
user_id	INT(11)	NULL	Recipient user ID
title	VARCHAR(255)	NOT NULL	Subject of the notification
message	TEXT	NULL	Notification message content

related_id	INT(11)	NULL	ID related to the notification (e.g., document ID)
related_type	ENUM('profile','document','batch','system')	NULL, default 'profile'	Type of related item
type	ENUM('success','warning','error','info')	NULL, default 'info'	Severity/Context level
created_at	TIMESTAMP	NOT NULL, default current_timestamp()	Time notification was created
is_read	TINYINT(1)	NULL, default 0	Read status (0 = unread, 1 = read)

B.5 System Logs and Audit Trails

Table B.11 *Alumni Activity Log (alumni_activity_log)*

Column Name	Data Type	Constraint	Description
log_id	INT(11)	PK, AUTO INCREMENT, NOT NULL	Unique activity record ID
action_type	VARCHAR(50)	NOT NULL	Categorization of user action
description	TEXT	NOT NULL	Detailed narrative of event
user_id	INT(11)	NULL	Link to users table
created_at	TIMESTAMP	NOT NULL	Timestamp when the action occurred

Table B.12 *Administrative Update Log (update_log)*

Column Name	Data Type	Constraint	Description
log_id	INT(11)	PK, AUTO INCREMENT	Unique update record ID.
updated_by	INT(11)	FK	Admin who performed the action
update_type	ENUM('update','approve','reject')	NULL	Admin action
update_details	TEXT	NULL	Link to users table

Appendix C: Software Performance and Data Loading Documentation

This appendix documents the technical execution of system performance testing and database population. The process utilized automated PowerShell scripts, PHP script execution, and SQL query within the XAMPP environment to incrementally seed the environment from an initial 1,000-record batch to a peak capacity of 200,000 records. This ensures that the system remains performant under extreme data conditions.

C.1 Database Loading

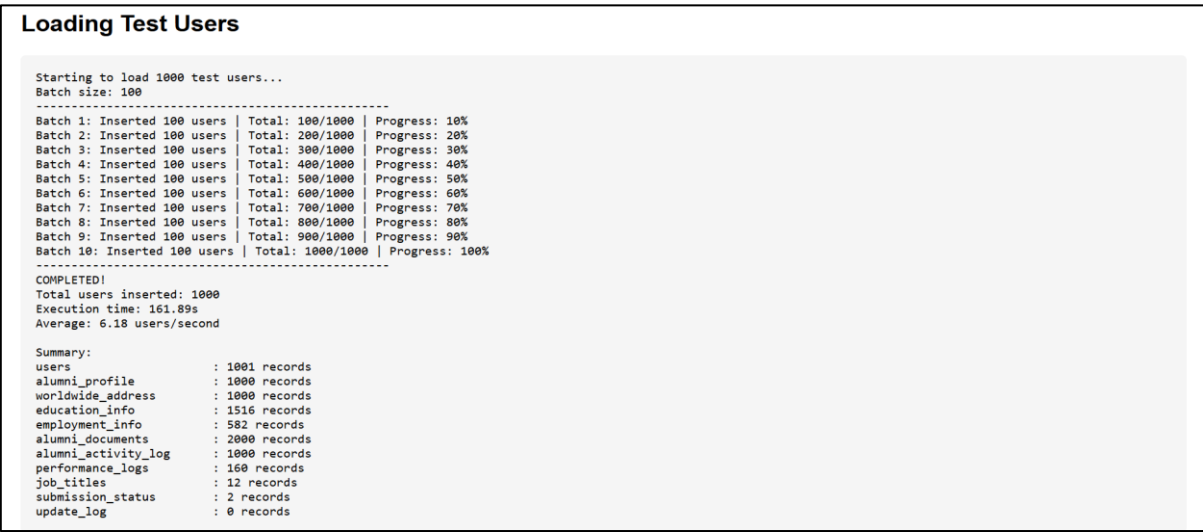


Figure C.1. Initial Records Loading (1,000 Records)

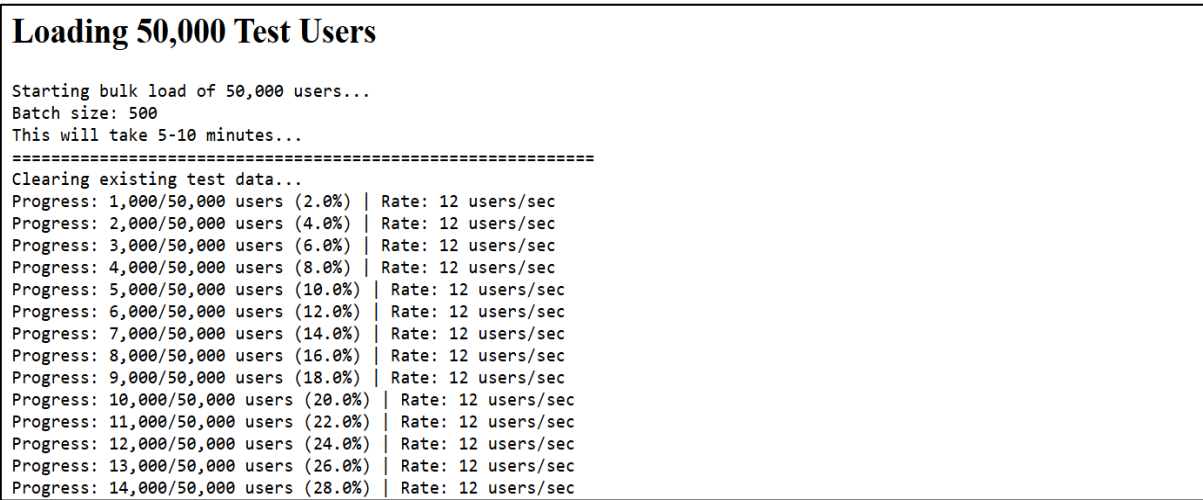


Figure C.2. High-Volume Bulk User Injection (50,000 Records)

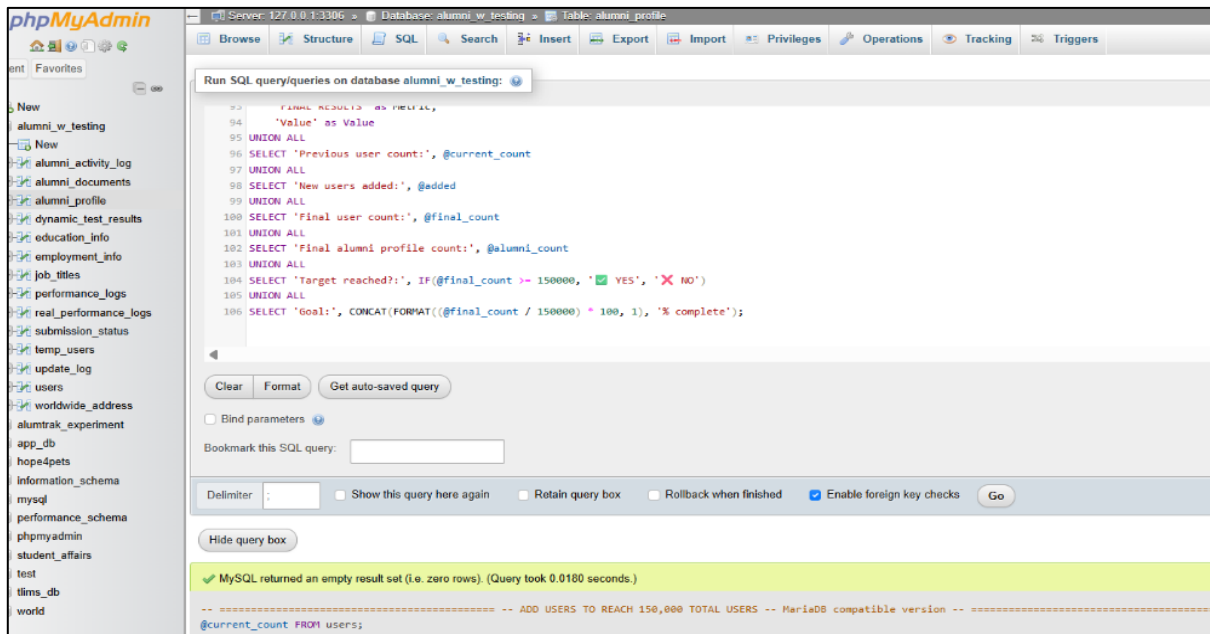


Figure C.3. Load Injection and System Scaling (150,00 User Records)

C.2 Performance Testing Execution

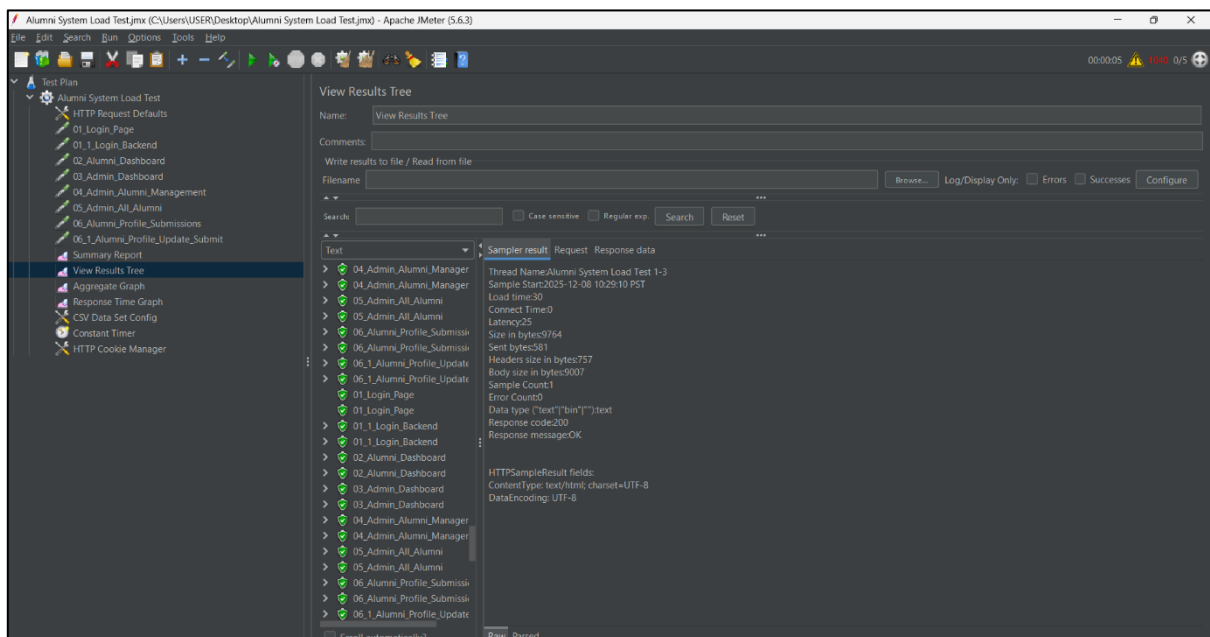


Figure C.4. Load Test Performance Summary

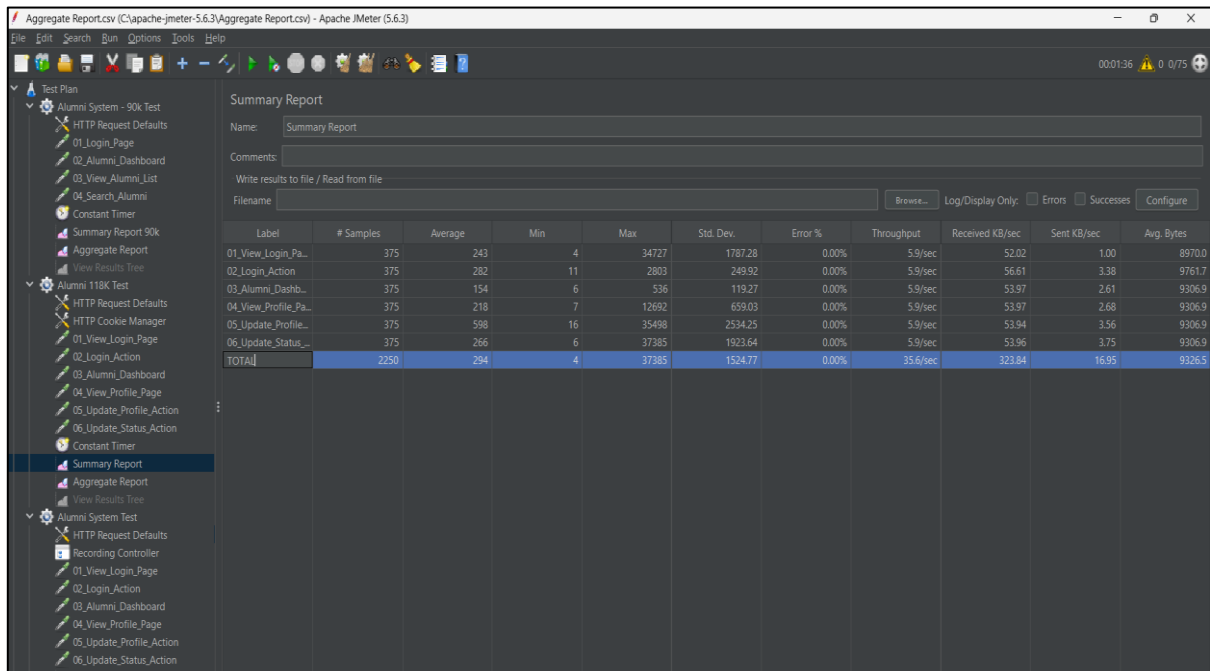


Figure C.5. Stress Test Performance Summary

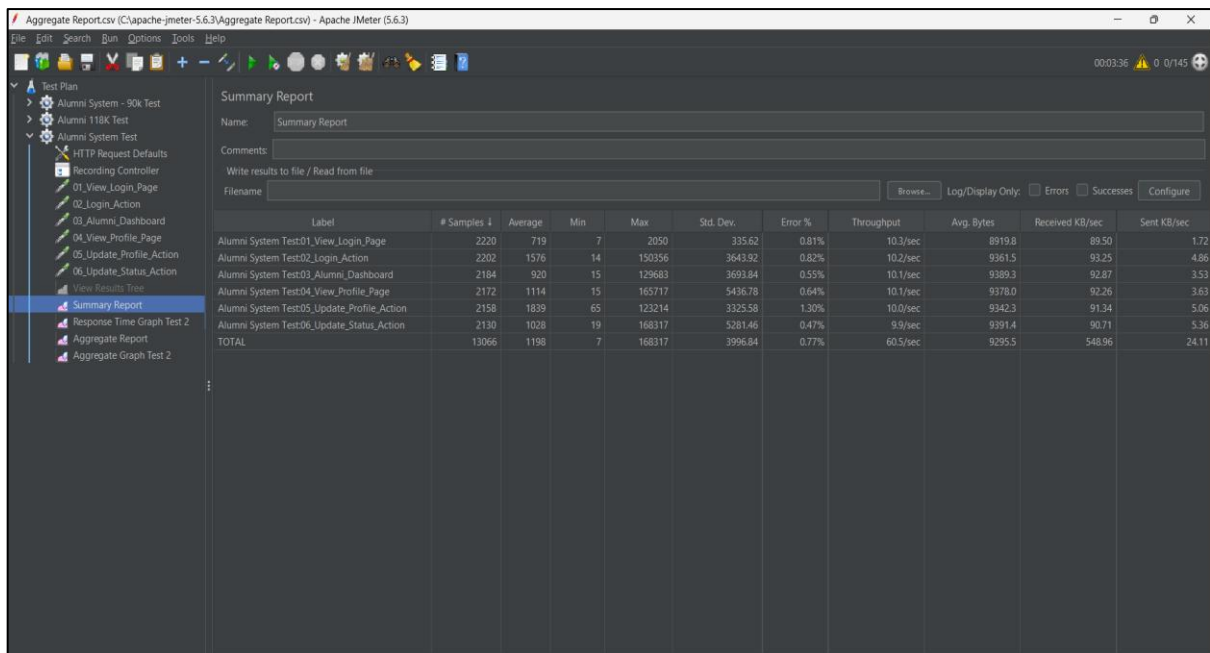
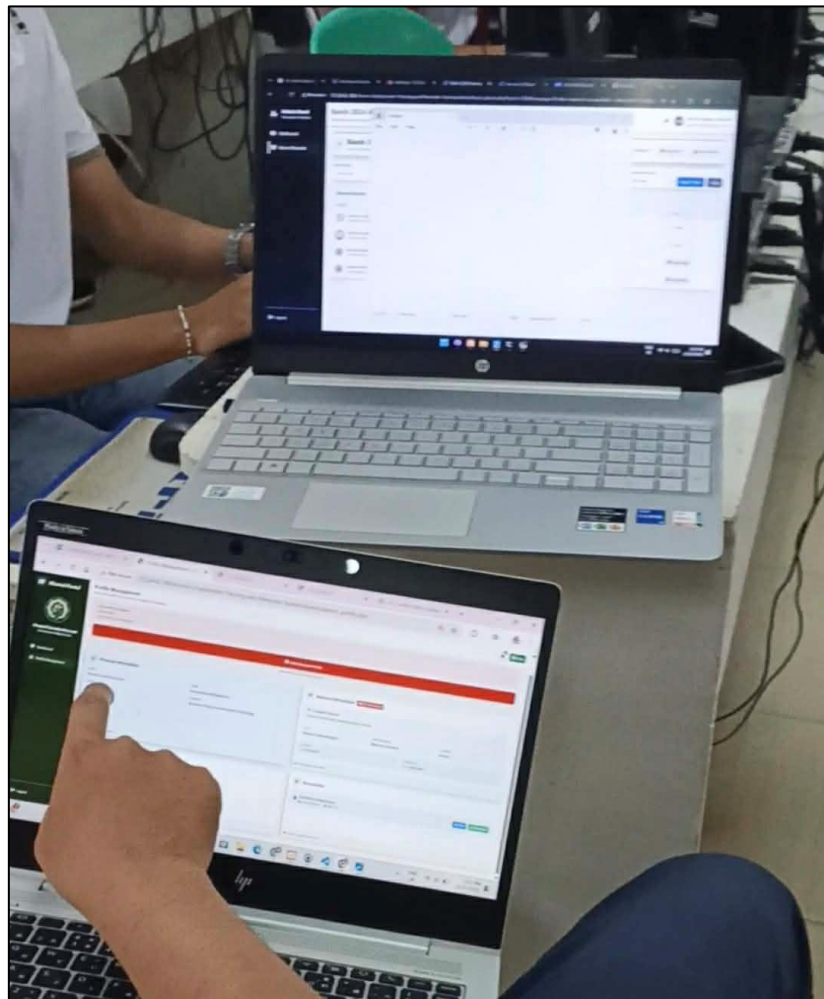


Figure C.6. Stress Test Performance Summary (100 Concurrent Users)

Appendix D: Beta Test Logs and Photographic Evidence

This appendix provides a visual record of the user evaluation phase for the AlumTrak platform. It features selected visual documentation capturing the interaction between users and the system, validating the actual conduct of the User Acceptance Testing (UAT). These visuals are supplemented by consolidated beta test logs which highlight the iterative refinements made to the system's interface and backend modules based on real-world user performance and technical observations recorded during the testing period.





Appendix E: Documentation of Project Development and Defense

This appendix provides a chronological visual record of the AlumTrak project's evolution from its conceptual phase to a fully developed and completed system.

E.1 Project Roadmap, Technical Consultations, and Development Phase

This section records the iterative development process and technical consultations with the Project Instructor that guided the transition from abstract requirements to functional code. During this phase, the system's backend logic and frontend interfaces were refined to ensure a reliable and well-structured architecture.

			AUGUST				SEPTEMBER				OCTOBER				NOVEMBER				DECEMBER	
			W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2
			4 - 8	11 - 22	08-25 / 09-05	8 - 19	09-22 / 10-03	6 - 17	10-20 / 11-07	10 - 14	17 - 28	1 - 5	8 - 12							
	Task to Do	Deliverables/Consultations/Presentation																		
Project Proposal & Team Formation	Form groups (max 3 members). Brainstorm and select a project idea. Define the problem statement, objectives, target users, and features. Research similar existing systems.	Project Proposal Document. Initial system concept and scope. Project team roles assigned. Group Presentation	Completed																	
System Planning & Requirement Analysis	System Planning & Requirement Analysis. Gather user requirements (Through research). Create functional and non-functional requirements. Draft system architecture (components, modules).	Proposal regarding your idea. Requirement Specification Document. System Architecture Diagram.	Completed																	
GUI Prototyping	Design your Graphical User Interface	Presentation of your initial GUI			Completed															
Database Design	Design ERD or NoSQL schema. Define relationships and constraints. Create sample datasets for testing.	Present the Database Schema Diagram/ERD Diagram. SQL or NoSQL structure file.				Completed														
Back-end Setup	Set up development environment of your back-end.	Presentation of your complete Back-end					Completed													
Front-end Integration w/ Back-end	Connect front-end and back-end	Present working interface linked to database.						Completed												
External API / Technology Integration	Integrate at least one external service (e.g., Google Maps, payment gateway, IoT sensor, email/SMS).	Presentation of integrated API/technology. Documentation on integration process.							Completed											
Security Implementation (if any)	Apply authentication (login, user roles). Implement authorization (access control). Validate and sanitize all inputs.	Present your secure login and role-based access. Code snippets showing security measures (if applicable).									Completed			Completed						
System Testing & Debugging	Conduct testing, integration testing, and user acceptance testing (in the school testing only). Fix bugs and optimize performance.	Testing Report. List of fixed issues. Comment and suggestion by the user tester.											In progress							
Finalization & Documentation	Complete user manual and developer documentation (SRS). Prepare presentation and demonstration video with actual presentation of the system.	Present fully functional system. Final Documentation.																In progress		
Project Presentation & System Defense	Present the system to panel. Demonstrate features. Answer technical and conceptual questions from evaluators.	Presentation Slides. Live system demo. Source code submission.																	Not started	
DEADLINE / PRESENTATION			Aug 9	Aug 23	Sep 6	Sep 20	Oct 4	Oct 18	Nov 8	Nov 15	Nov 29	Dec 6	Dec 13							



E.2 Final System Defense Documentation

